

Ch10 数组和指针

对应《C Primer Plus（第6版）》第 10 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：多维数组深度用法、`int (\*p)[5]` 这类复杂声明

10.0 本章定位与取舍

学完这一章你会写什么：用数组存一组数据、用指针遍历它、写一个能改到"外面数组"的函数。

必看：10.2 数组名不是变量 · 10.5 为什么 `a[i]` 等于 `*(a+i)` · 10.7 函数传数组

可跳过：10.6 的 `*p++`（第一次读知道有这么回事就行，写代码时回头再看）

以后再看：多维数组、指向数组的指针、动态内存（Ch12 会讲 `malloc`）

本章在课程里的位置：Ch4 埋过一句"数组名是地址"的伏笔，这一章兑现它；Ch9 函数与本章合用，才能写出"能改到外面"的函数。

10.1 先看一个真实场景：ADC 采样

单片机每隔一段时间读一次 ADC（模数转换），把 12 位的结果存起来：

```
/* 想象中的数据长这样，不是要你现在写 */
第 1 次采样：2048
第 2 次采样：2011
第 3 次采样：2056
...一共采 64 次
```

如果只用单个变量，你得声明 `adc0` 到 `adc63` 六十四个名字——这显然不行。

你需要的是「**一排连续的、同类型的格子**」，能用一个名字加编号访问。这就是**数组**。

10.2 数组：一排连续的格子

`int a[5];` 一次申请 5 个 `int` 的格子，用 `a[0]` 到 `a[4]` 访问。**下标从 0 开始，不是 1**——第一个格子叫 `a[0]`。

```
/* ch10_01_array.c */
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    int i;
```

```

    for (i = 0; i < 5; i++)
        printf("a[%d] = %d\n", i, a[i]);
    return 0;
}

```

编译: `gcc -std=c11 -Wall -Wextra -o a.exe ch10_01_array.c` | 运行输出 (实测):

```

a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50

```

新手坑 · 基础：越界不报错。 `a[5]` 编译器一句话都不说。C 不检查下标，它只是“悄悄读/写了数组后面那块内存”。轻则读到垃圾值，重则改坏别的变量。写循环边界时多看一眼：`i < 5` 不是 `i <= 5`。

10.3 数组名是地址，但不是变量

这一节是本章最容易学错的地方。先记结论，再看三组实验：

一句话模型：数组名是地址，但它不是变量——不能给它赋值，也不能自增。

实验一：数组名就是首格地址

```

/* ch10_02_addr.c */
#include <stdio.h>

int main(void)
{
    int a[3] = {7, 8, 9};

    printf("a      = %p\n", (void*)a);
    printf("&a[0] = %p\n", (void*)&a[0]);
    printf("&a[1] = %p\n", (void*)&a[1]);
    return 0;
}

```

运行输出 (实测，地址每次运行会变，看差值)：

```

a      = 0000005445BFF964
&a[0] = 0000005445BFF964  ← 和 a 完全相同
&a[1] = 0000005445BFF968  ← 比 a[0] 大 4 = sizeof(int)

```

实验二：sizeof 在 main 里和函数里不一样

这是"数组名不是指针"最直接的证据。同一个 a，出了 main 就"变小"了：

```
/* ch10_02_sizeof.c */
#include <stdio.h>

void show(int arr[])
{
    printf("函数里 sizeof(arr) = %zu\n", sizeof(arr));
    printf("函数里 sizeof(*arr) = %zu\n", sizeof(*arr));
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};

    printf("main 里 sizeof(a) = %zu\n", sizeof(a));
    printf("main 里元素个数 = %zu\n", sizeof(a) / sizeof(a[0]));

    show(a);
    return 0;
}
```

运行输出（实测）：

```
main 里 sizeof(a) = 20    ← 5 个 int，整个数组
main 里元素个数 = 5
函数里 sizeof(arr) = 8    ← 只是指针的大小（64 位）
函数里 sizeof(*arr) = 4   ← 一个 int
```

编译器会主动提醒你这件事。上面这段代码用 -Wall -Wextra 编译时，gcc 会报一条警告（实测原文）：

```
warning: 'sizeof' on array function parameter 'arr' will return size of 'int *' [-Wsizeof-array-argument]
```

这条警告就是编译器在替你回答"参数到底是不是数组"。所以：函数里永远不要用 sizeof(arr) 求数组长度，要把长度当参数传进去（见 10.7）。

实验三：给数组名自增，编译器直接拦下

❌ 错例（不要照抄）

```
/* debug_ch10_arrayname.c */
int a[3] = {1, 2, 3};
a++;           /* 想让 a 往后走一格 */
```

编译报错（实测原文）：

✅ 对例：让指针走

```
/* 想移动就用指针 */
int a[3] = {1, 2, 3};
int *p = a;
p++;           /* p 现在指向 a[1] */
```

```
error: lvalue required as
increment operand
```

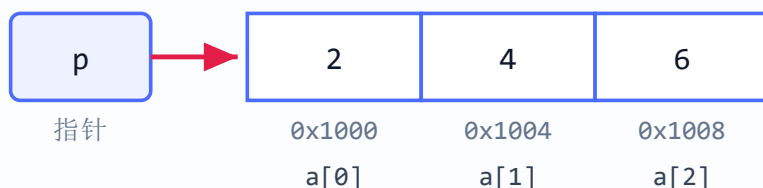
人话翻译：`++` 要求左边是"能被赋值的变量"，而数组名不是。

指针 `p` 是变量，可以自增；数组名 `a` 不是。

新手坑 · 基础：别把"数组名"当成"指针"。它们的值常常相同（都指向首格），但性质不同：数组名不能被赋值、不能自增、`sizeof` 是整个数组的大小；指针这三件事都能做。

10.4 指针：专门存地址的变量

`p` 里存的是 `a` 的地址



内存模型：数组 `a` 是一排连续的格子，地址 0x1000 / 0x1004 / 0x1008（每个 `int` 占 4 字节）；指针 `p` 自己也是一块内存，里面存着 0x1000 这个地址值。（地址为示意，实际每次运行不同）

两个配套运算符，C 里最基础也最容易搞反的一对：

- **&（取地址）**：`p = &n`；把 `n` 的地址存进 `p`。
- ***（解引用）**：`*p` 表示"`p` 指向的那个变量"。`*p = 5`；等价于给 `n` 赋值 5。

```
/* ch10_03_pointer.c */
#include <stdio.h>

int main(void)
{
    int n = 42;
    int *p = &n;    /* p 存 n 的地址 */

    printf("n = %d\n", n);
    printf("*p = %d\n", *p);    /* 通过指针读 n */
    *p = 100;                  /* 通过指针改 n */
    printf("改后 n = %d\n", n);
    return 0;
}
```

运行输出（实测）：

```
n = 42
*p = 42
改后 n = 100
```

新手坑 · 基础：野指针。 `int *p; *p = 5;` —— `p` 没初始化，指向哪里未知，往里写就是往一块不知道是谁的内存里写。

实测（gcc 16.1.0）：单独编译这一行会得到 warning: 'p' is used uninitialized [-Wuninitialized]（本机开不开 `-O2` 都会报）。

但不要用"有没有警告"来判断野指针——如果同一段代码里还有别的硬错误，编译器可能根本走不到这条分析。

10.5 `a[i]` 就是 `*(a+i)`：为什么

本章最核心的一个等式：

```
a[i] == *(a + i)
```

它不是约定，是定义。理解它需要先回答一个问题：`a + 1` 到底加了多少？

先看实验：同一个 +1，跳的距离不一样

```
/* ch10_06_ptrstep.c */
#include <stdio.h>

int main(void)
{
    int a[3] = {0x11111111, 0x22222222, 0x33333333};
    int *pi = a;
    char *pc = (char *)a;

    printf("int* pi + 1 = %p 跳 %ld 字节\n",
           (void *)(pi + 1), (long)((char *)(pi + 1) - (char *)pi));
    printf("char* pc + 1 = %p 跳 %ld 字节\n",
           (void *)(pc + 1), (long)((pc + 1) - pc));
    return 0;
}
```

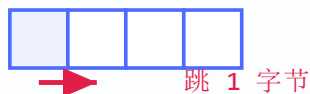
运行输出（实测，地址每次运行会变，看“跳几字节”）：

```
int* pi + 1 = 000000299C3FFE08 跳 4 字节
char* pc + 1 = 000000299C3FFE05 跳 1 字节
```

int* +1



char* +1



指针加法按它指向的类型缩放：int* 跳 `sizeof(int)=4` 字节，char* 跳 1 字节。所以 `a + i` 是"从数组开头往后数 `i` 个元素"的地址，不是"`i` 个字节"。

于是等式就成立了

1 a 是首格地址。

2 `a + i` 是"往后数 `i` 个元素"的地址——因为是按元素缩放，所以正好落在 `a[i]` 上。

3 `*(a + i)` 就是"取那个地址上的值" = `a[i]`。

所以下标只是指针运算的语法糖：`a[i]` 是写给人类看的，编译器看到的就是 `*(a+i)`。理解这一点，你就同时理解了：为什么数组名能当指针用、为什么指针也能用 `p[i]` 下标。

```
/* ch10_04_equiv.c */
#include <stdio.h>

int main(void)
{
    int a[4] = {2, 4, 6, 8};
    int *p = a;

    printf("a[2] = %d, *(a+2) = %d\n", a[2], *(a + 2));
    printf("p[2] = %d, *(p+2) = %d\n", p[2], *(p + 2));
    return 0;
}
```

运行输出（实测）：

```
a[2] = 6, *(a+2) = 6
p[2] = 6, *(p+2) = 6
```

10.6 *p++ 还是 (*p)++

这两个写法只差一对括号，但做的事**完全不同**。先看实测，再看记忆法：

```
/* ch10_07_ptrincr.c */
#include <stdio.h>

int main(void)
{
    int a[3] = {10, 20, 30};
    int *p = a;
    int x, y;

    x = *p++;    /* 先取出 *p, 再让 p 往后走一格 */
    printf("x = %d, *p = %d, p 指向 a[%ld]\n", x, *p, (long)(p - a));

    p = a;      /* 复位 */
    y = (*p)++; /* 先取出 *p, 再把那一格内容加 1; p 不动 */
    printf("y = %d, a[0] = %d, p 仍指向 a[%ld]\n", y, a[0], (long)(p - a));
    return 0;
}
```

运行输出（实测）：

```
x = 10, *p = 20, p 指向 a[1]
y = 10, a[0] = 11, p 仍指向 a[0]
```

写法	谁变了	变多少	结果
*p++	指针 p	往后走 1 个元素	遍历数组用
(*p)++	p 指向的内容	内容 +1	给数组元素计数累加用

记忆法：看 ++ 贴着谁。++ 紧贴 p（*p++ 里的 p++）就动指针；++ 紧贴括号 ((*p)++) 就动内容。

状态追踪：用 *p++ 遍历数组

把每一步的变量值摊开来看，比读文字清楚得多：

```
/* ch10_08_trace.c 的循环体 */
for (i = 0; i < 4; i++)
    sum += *p++; /* 取值 + 指针前进，一步完成 */
```

轮次 i	执行前 *p	执行前 p 指向	sum 累加后	执行后 p 指向
0	2	a[0]	2	a[1]
1	4	a[1]	6	a[2]

2	6	a[2]	12	a[3]
3	8	a[3]	20	a[4]（末尾之后）

运行输出（实测）：

```
sum = 20
循环结束后 p - a = 4（p 已走到末尾之后再一格）
```

新手坑 · 基础：循环结束后 p 已经不在数组里了。 $p - a = 4$ ，而数组只有 $a[0]$ – $a[3]$ 。此时**不要解引用** $*p$ 。规范的写法是在循环里用 $a[i]$ ，或者循环后重新 $p = a$ ；。

10.7 函数传数组：传的是地址

函数形参写成 `int arr[]`，实际拿到的是**指向数组开头的指针**——数组不会被复制。所以函数里改数组，外面的数组会跟着变。

```
/* ch10_05_func.c */
#include <stdio.h>

int sum(const int arr[], int n) /* const 承诺不修改数组内容 */
{
    int i, total = 0;
    for (i = 0; i < n; i++)
        total += arr[i];
    return total;
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    printf("总和 = %d\n", sum(a, 5));
    return 0;
}
```

运行输出（实测）：

```
总和 = 15
```

为什么 n 必须单独传？ 回到 10.3 的实验二：函数里 `sizeof(arr)` 只有 8，量不出数组有多长。所以「数组 + 长度」要成对传，这是 C 的固定套路。

为什么写 const？ 函数不打算改数组时就写 `const int arr[]`，编译器会拦住你误改——这是给自己设的护栏。

10.8 新手坑：基础与扩展

级别	坑	怎么躲
基础	下标越界 (<code>a[5]</code> 合法访问 5 个元素的数组)	循环写 <code>i < n</code> ; C 不检查越界
基础	函数里 <code>sizeof(arr)</code> 当数组长度	长度当参数传; 看 <code>-Wsizeof-array-argument</code> 警告
基础	给数组名自增 / 赋值	要移动就用指针; 数组名不是变量
基础	<code>&</code> 与 <code>*</code> 搞反	<code>p = &n</code> 存地址; <code>*p = 5</code> 改内容
基础	以为传数组是复制	函数改形参数组 = 改外面; 要副本得自己拷
扩展	循环结束后解引用 <code>*p</code>	<code>p</code> 已到末尾之后; 循环后重新 <code>p = a</code>
扩展	把 <code>a</code> 和 <code>&a</code> 当成一回事	值相同但类型不同 (<code>&a+1</code> 跳整个数组)

10.9 一分钟自测（先答后揭）

先在纸上写下你的答案，再点击对照——**直接看答案等于没做。**

▼ ① `int a[5] = {1, 2, 3, 4, 5}; printf("%d", a[5]);` 会输出 5 吗?

正解：不会。`a[5]` 越界了——合法下标只有 0–4。它读到的是数组后面那块内存，值是**不确定的**（可能正好是别的变量，也可能是垃圾值）。

常见错解："会，因为 `a[5]` 是第 5 个元素。" ← 错在把"第 5 个"当成 `a[5]`；第一个元素是 `a[0]`，第 5 个是 `a[4]`。

为什么这条最容易错：日常语言从 1 数起，C 从 0 数起；而且越界**不报错**，写错的人常常"看起来能跑"。

▼ ② `int *p; *p = 10;` 是合法写法吗?

正解：语法合法，但行为未定义——不能这么写。`p` 没初始化，里面是残留值，`*p = 10` 等于往一块不知道属于谁的内存里写。

常见错解："编译能过就是合法。" ← 错在混淆"编译通过"与"行为正确"。C 里这两件事经常不一致，这也是本章反复强调"看编译器警告"的原因。

正确做法：先让指针有明确指向 (`int n; int *p = &n;`)，或指向动态内存 (Ch12)。

▼ ③ 函数形参写 `int arr[]`，它实际是什么类型？`sizeof(arr)` 是多少?

正解：实际是 `int *arr`；`sizeof(arr)` 是指针大小（本机 64 位 = 8），不是整个数组。

常见错解："是数组，`sizeof` 应该等于元素个数 $\times 4$ 。" ← 错在把形参当成了数组本体。数组名作实参时

会"退化"成指向首元素的指针，函数拿到的是地址，不是副本。

交叉验证：本机实测 main 里 `sizeof(a)=20`、函数里 `sizeof(arr)=8`（见 10.3 实验二）。

10.10 练习指引

四级阶梯，按顺序做（做完再去练习册对答案）：

1. **照抄改数：**把 `ch10_01_array.c` 的初值改成你自己的 5 次 ADC 采样值，打印出来。
2. **自己写逻辑：**写一个函数 `int max_of(const int arr[], int n)` 返回最大值，并在 main 里调用验证。
3. **组合两个知识点：**用 10.6 的 `*p++` 写法重写上面的遍历，并对照 `p - a` 观察指针最终停在哪。
4. **专业场景：**用数组存 64 次 12 位 ADC 采样，算出平均值与峰峰值（最大值 - 最小值）——先想清楚"为什么函数里量不出数组长度"。

本章配套练习见《练习册 · Ch10》。主教材复习题建议完成 **2、4、6** 三题。

下一章预告：Ch12 会讲动态内存 `malloc/free`——那是"数组长度在运行时才确定"时的办法。本章的 `a[i] == *(a+i)` 与 `p - a` 在那里会再用到。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均用 `gcc -std=c11 -Wall -Wextra` 编译并实跑，输出与文中标注一致（唯一例外见 10.3 实验二，那条警告是本节的教学内容本身）。原书题目请以正版教材为准。